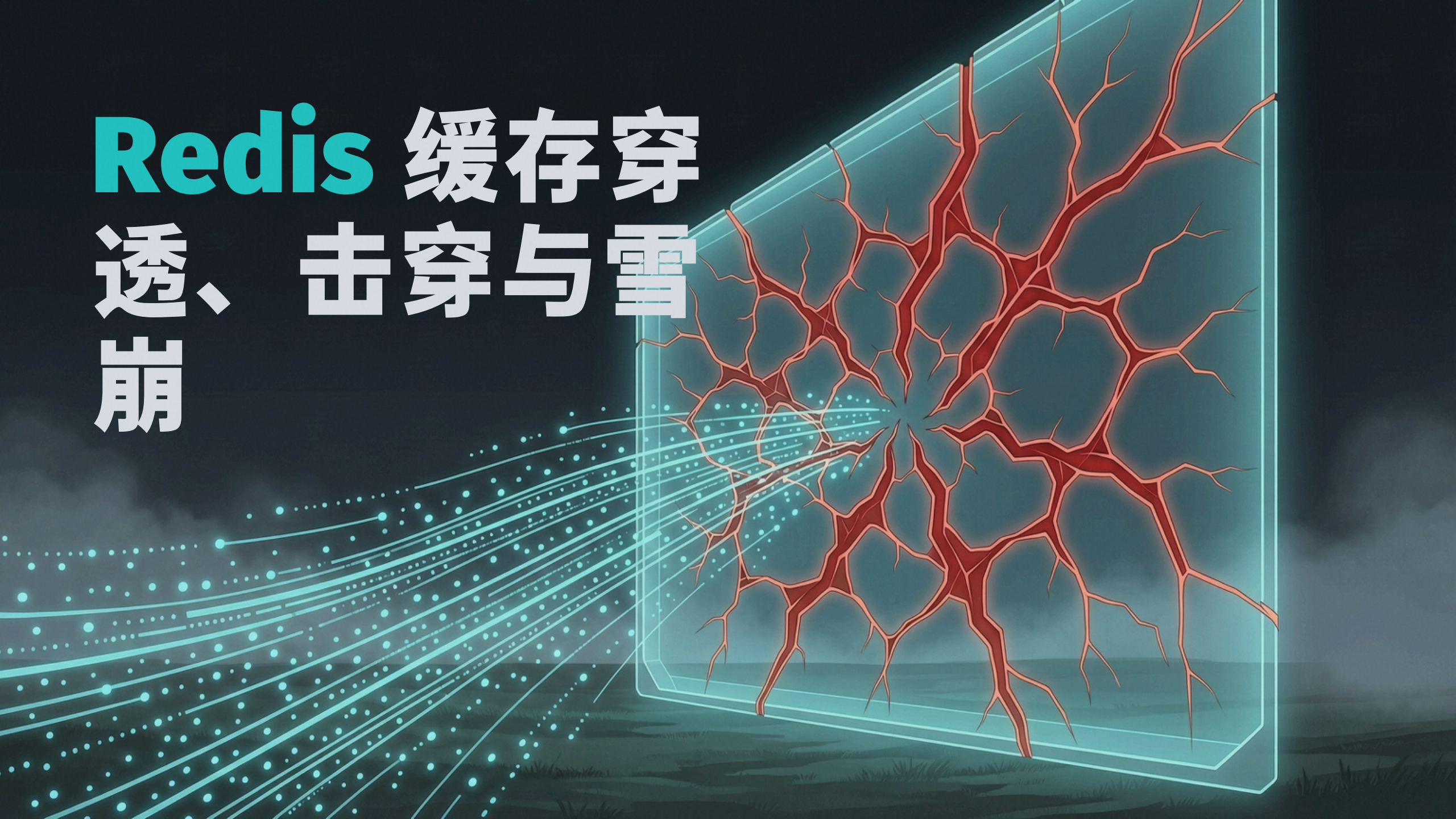
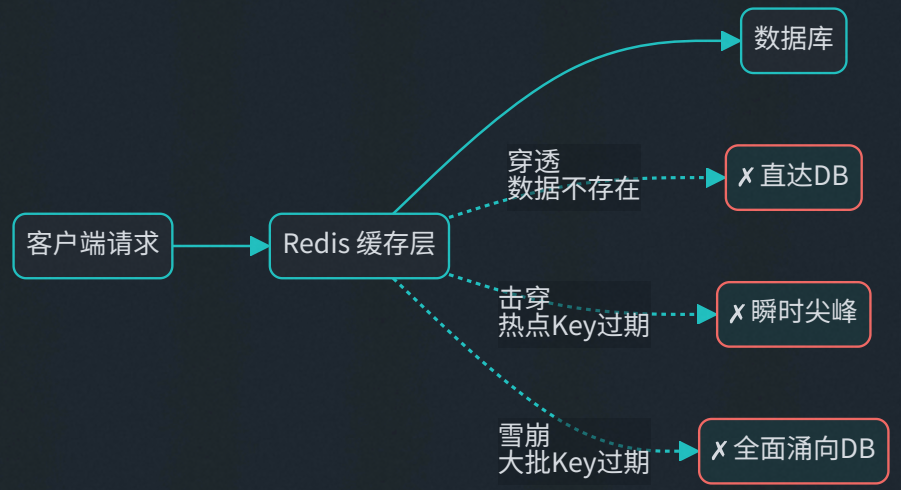
Redis 缓存穿透、击穿与雪崩



缓存保护层的三种失效方式

在请求与数据库之间，缓存是第一道保护层。正常情况下，绝大多数读请求在 Redis 这一层就被返回，数据库几乎无感。一旦保护层失效，流量会直接打到数据库上——但失效的方式有三种，机制完全不同。



穿透

数据根本不存在于缓存和DB，请求每次都穿透缓存直达数据库，持续消耗DB资源

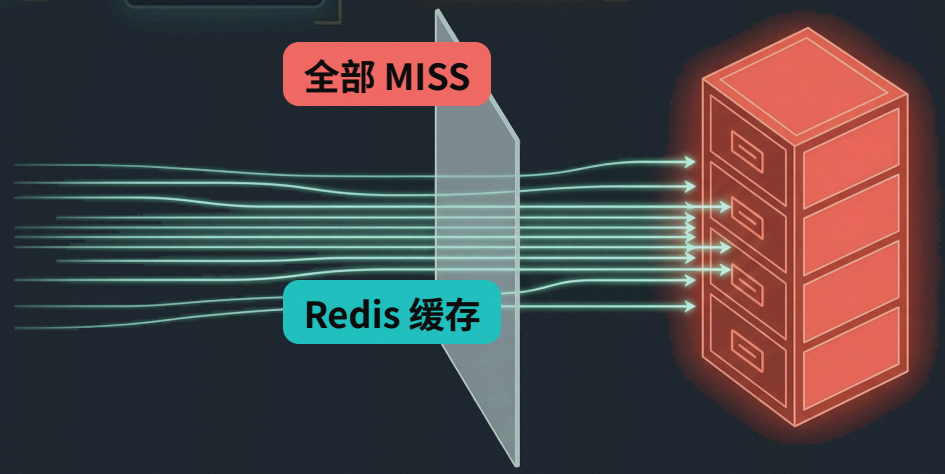
击穿

数据存在，但某个被高频访问的热点Key在过期瞬间，大量并发同时回源，形成瞬时尖峰

雪崩

大批Key在同一时刻集中过期，或Redis实例整体宕机，缓存层大面积失效，流量全面涌向DB

缓存穿透：查询根本不存在的数据



穿透：缓存层形同虚设，所有请求直达数据库

客户端请求一个 Key，缓存未命中，去 DB 查——DB 也没有。因为 DB 没有结果，缓存里自然也不会写入任何东西。

下一次同样的请求再来，**依然缓存未命中、依然查 DB**，如此循环。如果请求量足够大，数据库就被无效查询持续消耗。

穿透 vs 击穿：本质区别

穿透

数据**根本不存在**，无论缓存还是 DB 都没有，请求永远无法命中

vs

击穿

数据存在，只是缓存**恰好在此刻过期**，并发集中在单个热点 Key

典型触发场景

- **恶意攻击**：黑客构造海量不存在的 ID（如 user_id=-1 或随机 UUID），每次都穿透到 DB
- **校验缺失**：前端或网关未做合法性校验，非法参数直达业务层，查 DB 得到空结果
- **爬虫遍历**：爬虫遍历非连续主键空间，大量请求命中不存在的记录

穿透的防御：布隆过滤器 + 缓存空值

两种方案叠加：布隆过滤器做前置拦截，缓存空值做兜底

方案一 布隆过滤器

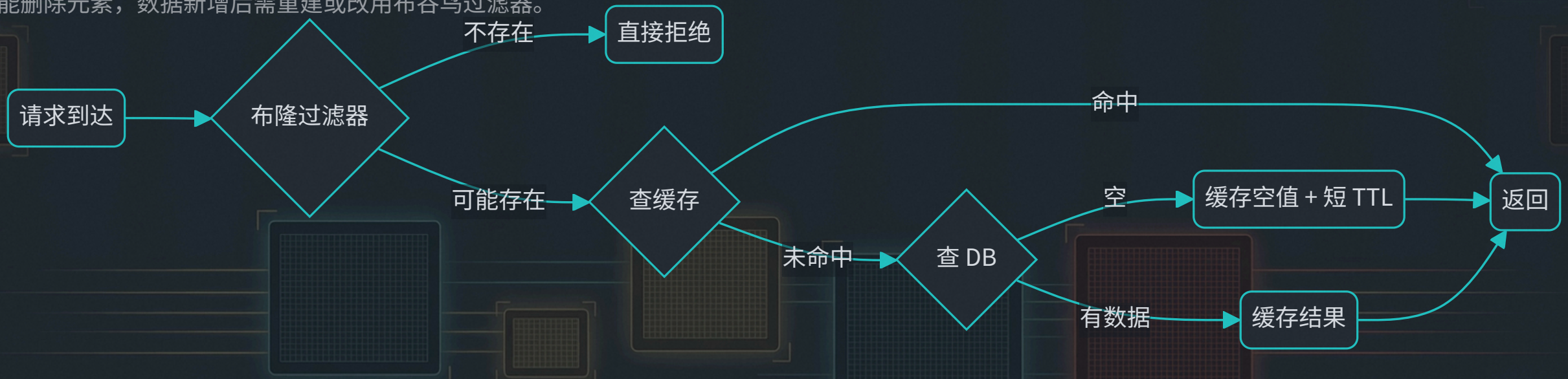
位数组 + 多个哈希函数。返回"不存在"→ 数据一定不存在（100% 准确）；返回"可能存在"→ 需进一步查缓存或 DB（存在误判但不会漏判）。存 100 万 ID、误判率 ≤1% 时仅需约 9.6MB，查询 O(k)。适用于写少读多的前置拦截。

取舍：内存占用极小（1 亿条约 100MB），拦截效率高；但存在误判率，不能删除元素，数据新增后需重建或改用布谷鸟过滤器。

方案二 缓存空值

查询 DB 返回空时，仍将该 Key 写入 Redis，value 设为特殊标记如 "NULL"，设置较短 TTL（30~300 秒）。同一 Key 的重复请求会命中空值缓存，不再穿透到 DB。

取舍：实现简单，无需额外组件；但治标不治本，海量不同无效 ID 仍会消耗缓存空间，需配合短 TTL 和容量监控。



缓存击穿：单个热点 Key 过期的瞬间

击穿的本质是**数据存在**，但缓存恰好在高并发时刻过期，瞬时流量全部回源到数据库



某个被高频访问的**热点Key**，在过期的那一瞬间，大量并发请求同时发现缓存未命中，全部回源到数据库。数据是存在的，只是缓存恰好在这个时刻失效了——并发集中在单个Key上，形成**瞬时尖峰**。

典型触发场景

热点商品 大促期间某个爆款SKU被数万用户同时访问，缓存过期瞬间DB承压

爆款文章 突发热点事件导致某篇文章阅读量暴增，缓存失效时并发回源

排行榜 全局共享的热点数据，过期时刻所有请求同时穿透

穿透 vs 击穿：本质区别

穿透

数据根本不存在，无论缓存还是DB都没有，请求永远无法命中，持续消耗DB

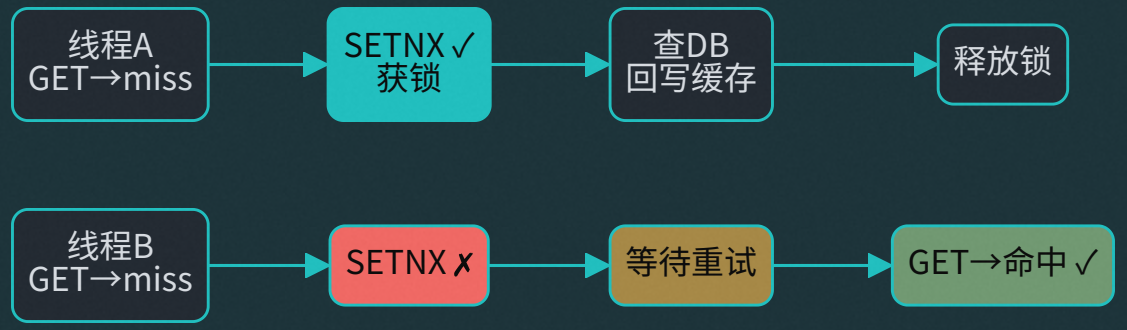
击穿

数据存在，只是缓存恰好在这个时刻过期，并发集中在单个热点Key上，形成瞬时压力尖峰

击穿的防御：互斥锁与逻辑过期

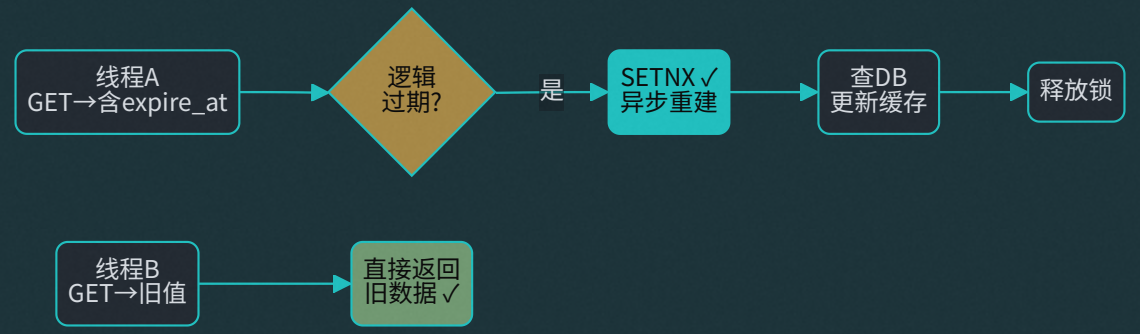
热点Key过期瞬间，不能让所有并发请求同时回源——两种方案分别从**一致性**和**可用性**两个维度解决

互斥锁 SETNX　强一致 · 牺牲可用性



只一个线程获锁查DB并回写缓存，其余**等待重试**。优化：随机退避、Watch Dog续期、客户端ID防误删。

逻辑过期　高可用 · 牺牲强一致



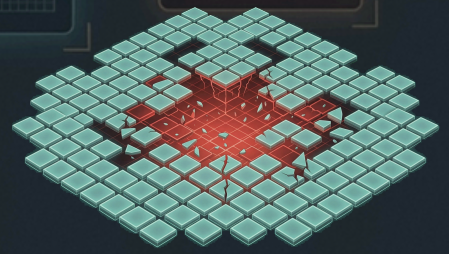
缓存不设实际TTL，value嵌入expire_at。过期后一个线程**异步重建**，其余返回旧数据。

互斥锁 强一致 · 牺牲可用性 · 成本低 → 金融交易、库存扣减

逻辑过期 最终一致 · 高可用 · 成本中 → 商品详情、资讯展示

混合使用 核心字段互斥锁（库存）+ 辅助字段逻辑过期（评论数）

缓存雪崩：大批 Key 同时失效



雪崩与穿透、击穿不同——它是**面状问题**而非点状问题。两种触发条件：大量 Key 在同一时刻集中过期，或 Redis 实例整体宕机。防御需要分层。

触发条件

- **大量 Key 同时过期**：预热 100 万 key 都设 EX 3600，一小时后集中失效
- **Redis 整体宕机**：单点故障或网络分区导致缓存层完全不可用
- **热点数据集中失效**：同类业务数据 TTL 相同，同一窗口内批量过期

预防层

TTL 随机化：基础 TTL + 随机偏移量。如文章详情缓存 30 分钟 + 随机 0~10 分钟，最终 30~40 分钟。不适合验证码等需固定过期窗口的业务。

架构层

多级缓存 + 高可用：本地缓存（Caffeine）→ Redis → DB；Redis 集群（Sentinel/Cluster）保证高可用。

兜底层

限流降级：DB 扛不住时快速失败。Sentinel/Hystrix 限流，熔断保护，返回默认值或缓存兜底数据。

三者对比：一眼定位、对症下药

遇到线上问题时，先判断属于哪种情况：数据是否**存在**？是否集中在单个**热点Key**？是否**大批Key同时失效**？然后根据下表选择对应的防御方案。

	穿透	击穿	雪崩
问题本质	数据根本不存在	单个热点Key过期瞬间	大批Key同时失效或Redis宕机
影响范围	单个不存在的Key持续穿透	单个热点Key的并发请求	大批Key或整个缓存层
触发条件	请求不存在的数据	热点Key恰好过期	TTL相同集中过期 / Redis宕机
对DB的影响	持续消耗DB资源	瞬时压力尖峰	流量全面涌向DB
核心防御	布隆过滤器 + 缓存空值	互斥锁 / 逻辑过期	TTL随机化 + 多级缓存 + 限流降级